

Git 整体逻辑与命令速查

面向本仓库日常使用：本地开发 → 提交 → 推送 GitHub → 查看历史 → 回退

配套实践仓库：[my-projects](#)

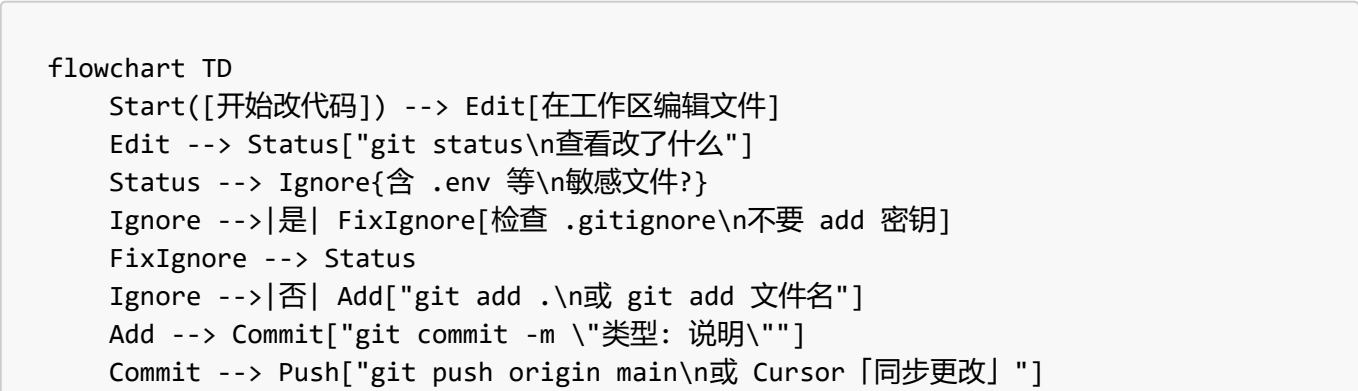
一、Git 是什么：四层存储架构

Git 不是「云盘」，而是**带历史的版本管理系统**。数据在四个区域之间流动：



区域	是什么	常用命令
工作区	磁盘上能直接编辑的文件	编辑器保存
暂存区	下次 commit 的「购物车」	<code>git add</code>
本地仓库	<code>.git</code> 里的完整历史	<code>git commit</code>
远程仓库	GitHub 上的副本	<code>git push / git pull</code>

二、一次完整日常流程（最常用）



```
Push --> Done([GitHub 可见新历史])

style Ignore fill:#fff3cd
style FixIgnore fill:#f8d7da
```

对应命令（复制即用）

```
cd "d:\job hunting"

git status                # 1. 看状态
git add .                 # 2. 暂存（或 git add 具体文件）
git commit -m "feat: 实现 chat_basic" # 3. 本地提交
git push origin main      # 4. 推到 GitHub
```

Cursor 中文界面对照

Git 命令	Cursor 操作
git status	源代码管理面板查看变更列表
git add	文件旁点 + 或暂存全部更改
git commit	输入说明 → 点 提交
git push	点 同步更改 (v)

三、项目生命周期：从 0 到 GitHub

```
flowchart TD
    subgraph 首次搭建
        A0[git config 用户名邮箱] --> A1[GitHub 网页 New repository]
        A1 --> A2["git init"]
        A2 --> A3[编写 .gitignore]
        A3 --> A4["git add ."]
        A4 --> A5["git commit -m \"init: ...\""]
        A5 --> A6["git remote add origin URL"]
        A6 --> A7["git push -u origin main"]
    end

    subgraph 日常循环
        B1[改代码] --> B2[add → commit → push]
        B2 --> B1
    end

    A7 --> B1
```

步骤	命令	说明
----	----	----

步骤	命令	说明
配置身份	<code>git config --global user.name "名"</code> <code>git config --global user.email "邮箱"</code>	只做一次
初始化	<code>git init</code>	当前目录变仓库
关联远程	<code>git remote add origin https://github.com/用户/仓库.git</code>	只做一次
首次推送	<code>git branch -M main</code> <code>git push -u origin main</code>	<code>-u</code> 绑定 upstream
之后推送	<code>git push</code>	已绑定后可直接 push

四、Commit 历史：如何「看」

flowchart TD

GH[GitHub 仓库首页] --> CL[点击「X 次提交」]

CL --> List[Commits 列表\n每条 = 一次快照]

List --> One[点进某次 commit]

One --> Diff[查看所有文件 diff\n绿增红删]

GH2[点进单个文件] --> Hist[History 历史记录]

Hist --> OneFile[该文件每次改动]

OneFile --> FileDiff[该文件在某 commit 的 diff]

GH3[Blame 追溯] --> Line[每一行是谁哪次写的]

目的	本地命令	GitHub 网页
提交列表	<code>git log --oneline</code>	Commits 页
某次改动详情	<code>git show <commit></code>	点进 commit
单文件历史	<code>git log -- 文件路径</code>	文件 → History
谁改了某行	<code>git blame 文件路径</code>	Blame

示例（你的仓库）：

ad3bbb4 init: Day 0 环境搭建与学习文档

0c7c1f3 docs: add root README ... ← 若存在则为第二次

五、回退：三种策略架构

flowchart TD

Q{目标是什么?}

```
Q -->|彻底回到某次提交\n不要之后的历史| R[git reset]
Q -->|保留历史\n用新提交撤销| V[git revert]
Q -->|只是看看旧版\n不删历史| S[git checkout]

R --> RH{"reset 类型"}
RH -->|工作区也还原| RHard["git reset --hard "]
RH -->|保留文件在暂存区| RSoft["git reset --soft "]
RH -->|保留文件未暂存| RMixed["git reset --mixed "]

RHard --> FP{已 push?}
FP -->|是| Force["git push --force\n⚠ 个人仓库可用"]
FP -->|否| LocalOnly[仅本地改变]

V --> VCmd["git revert "]
VCmd --> PushNormal[git push 正常推送]

S --> SCmd["git checkout -- 文件"]
```

5.1 场景对照表

场景	推荐命令	历史是否保留	远程
撤销最近一次 commit，保留文件	<code>git reset --soft HEAD~1</code>	删除该 commit	需 force push
回到第一次提交，文件也还原	<code>git reset --hard ad3bbb4</code>	删除之后 commit	需 force push
已 push，安全撤销 README 那次	<code>git revert 0c7c1f3</code>	保留 + 新增撤销 commit	正常 push
误操作救回	<code>git reflog → git reset --hard <hash></code>	可恢复	—

5.2 HEAD 与 HEAD~1

```
HEAD      → 当前所在 commit (最新)
HEAD~1    → 上一个 commit
HEAD~2    → 上上个 commit
```

六、分支（进阶，了解即可）

```
gitGraph
  commit id: "init"
  commit id: "add README"
  branch feature-agent
  checkout feature-agent
```

```
commit id: "WIP agent"
checkout main
commit id: "docs update"
merge feature-agent
```

命令	作用
git branch	列出分支
git checkout -b 新分支	创建并切换
git merge 分支名	合并到当前分支

现阶段建议： 个人学习仓库只在 main 上 commit + push 即可。

七、.gitignore 在流程中的位置

```
flowchart LR
    Edit[编辑 .env 等] --> Status[git status]
    Status --> GI{.gitignore\n已忽略?}
    GI -->|是| Hidden[不出现在变更列表\n不会被误提交]
    GI -->|否| Danger[会出现在 add 列表\n⚠ 检查后再 add]
```

两大作用：

- 1. 轻量 — 忽略 __pycache__、模型、日志等大文件
- 2. 安全 — 忽略 .env 密钥，避免推到 GitHub

提交前 habit：

```
git status
git check-ignore -v .env      # 确认 .env 被忽略
```

八、命令速查总表

8.1 日常

命令	作用
git status	工作区 / 暂存区状态
git add .	暂存所有改动
git add 路径	暂存指定文件
git commit -m "msg"	提交到本地仓库

命令	作用
<code>git push</code>	推送到远程
<code>git pull</code>	拉取并合并远程更新
<code>git log --oneline</code>	简洁历史
<code>git diff</code>	未暂存的改动
<code>git diff --staged</code>	已暂存、未 <code>commit</code> 的改动

8.2 远程

命令	作用
<code>git remote -v</code>	查看远程地址
<code>git clone URL</code>	克隆他人仓库
<code>git fetch</code>	只下载远程更新，不合并

8.3 撤销 / 回退

命令	作用
<code>git restore 文件</code>	丢弃工作区改动
<code>git restore --staged 文件</code>	从暂存区撤出
<code>git reset --soft HEAD~1</code>	撤销 <code>commit</code> ，保留暂存
<code>git reset --hard <commit></code>	硬回退到指定 <code>commit</code>
<code>git revert <commit></code>	新建 <code>commit</code> 撤销某次
<code>git reflog</code>	找回「丢失」的 <code>commit</code>

8.4 Commit 信息规范（本仓库）

```
init: 初始化
feat: 新功能
fix: 修 bug
docs: 文档
test: 测试
```

九、本仓库推荐 Git 节奏

```
flowchart LR
    D1[Day 1 功能完成] --> C1[commit feat]
    D7[Week 1 交付] --> C2[commit + push\ntag 可选]
```

D14[Week 2 交付] --> C3[commit + push]
D21[Week 3 作品集] --> C4[commit + push\n更新简历]
D28[Month 结束] --> C5[最终 push\nGitHub 展示完整历史]

- 每完成一个小功能 → 一次 **commit**
- 每周末 → **push** 到 GitHub
- push 前 → **git status** 确认无 **.env**

十、常见问题

问题	处理
<code>fatal: not a git repository</code>	<code>cd</code> 到项目根目录
<code>Permission denied (push)</code>	检查 GitHub Token / 登录
误 add 了 <code>.env</code>	<code>git restore --staged .env</code> , 确认 <code>.gitignore</code>
push 被拒绝 (远程有新提交)	<code>git pull --rebase origin main</code> 再 push
reset 后后悔了	<code>git reflog</code> 找 hash 再 reset 回去

文档版本: v1.0 | 位于 `md/git-工作流程与命令.md`